

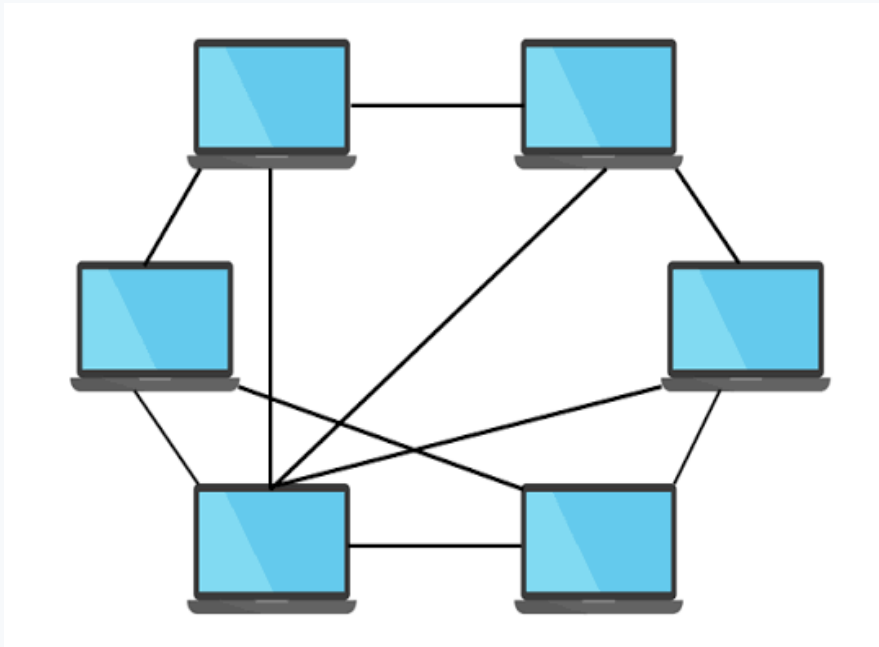
주변 친구

신선우

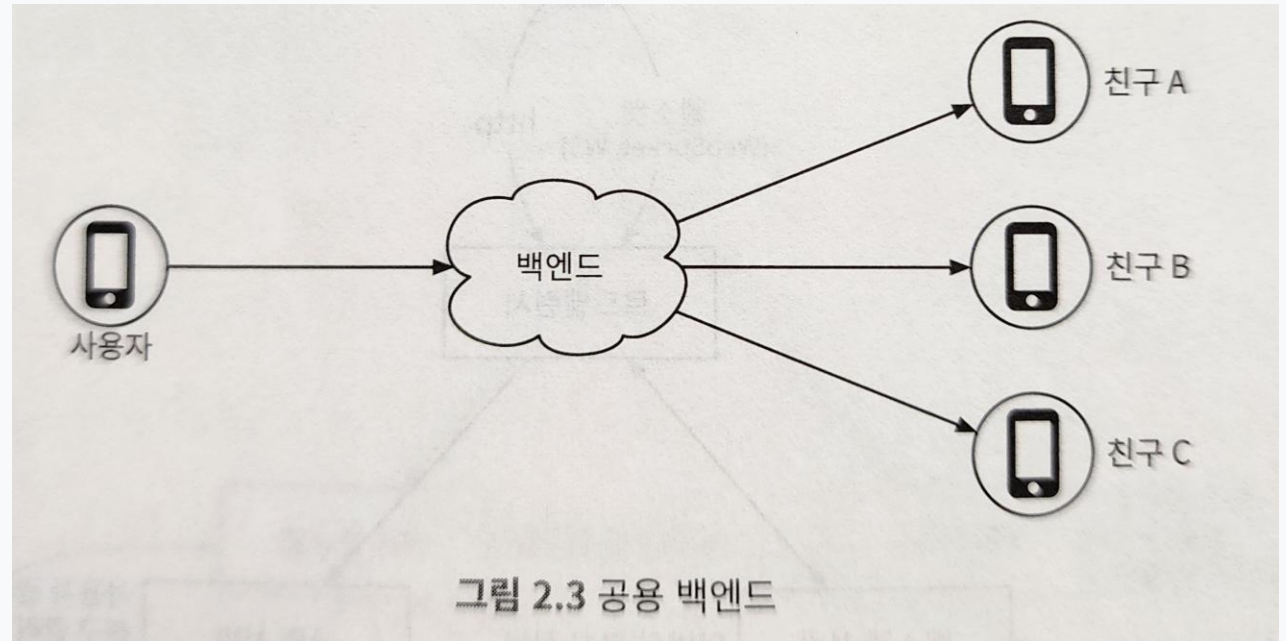
문제 설계

- '주변 친구'는 **5마일(8km) 반경 이내** 친구로 정의
- 친구 위치 정보는 **30초** 주기
- 하루 주변 친구 검색 기능 활용할 **사용자 1억명**으로 가정
- 동시 접속자 사용자 수는 10% 가정
 - $10\% * 1억 = 1000만 명$
- 평균적 한 사용자가 **400명의 친구**를 갖는다고 가정
- 이 기능은 **페이지당 20명**의 주변 친구를 표시

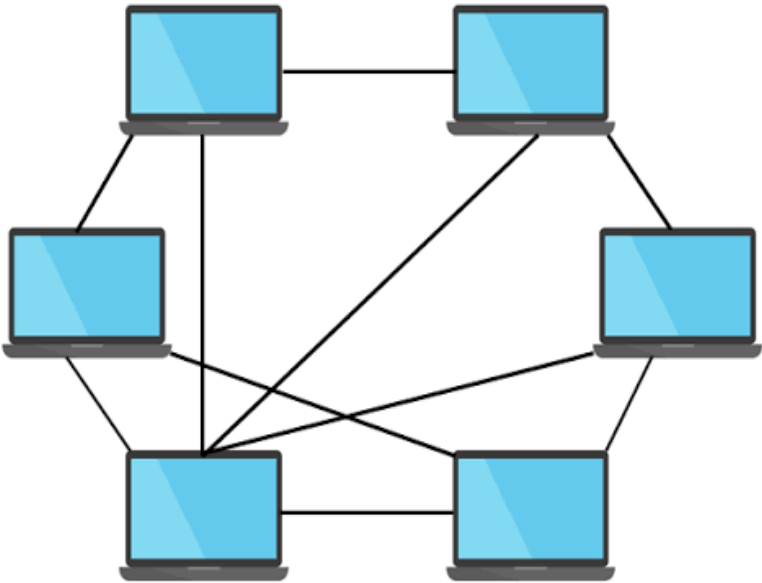
개략적 설계안



P2P 통신



P2P 통신



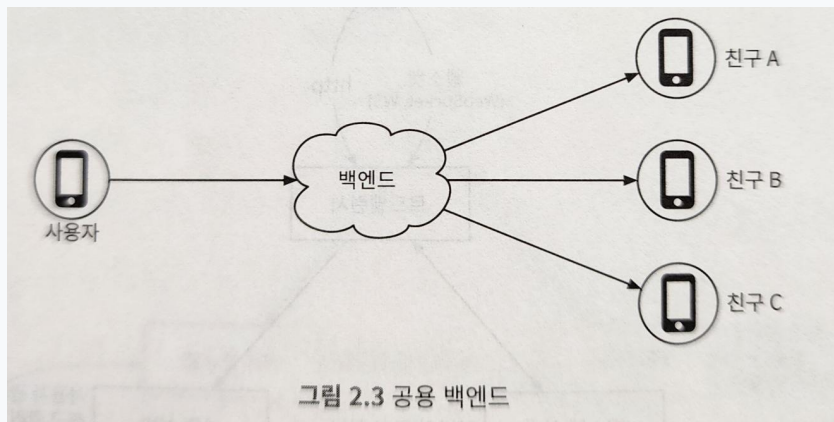
- 모바일 단말 통신 연결 상태 불량

- 전력 문제

- **탈락**

공용 백엔드

큰 규모에 적용하기 쉽지 않다.



활성 상태의 동시 접속 사용자가 천만 명 정도
위치 정보를 30초마다 갱신하면
초당 334,000번 위치 정보 갱신

- $\text{TPS} : 10,000,000 \text{ 트랜잭션} / 30\text{초} = 3,333,333.33333\text{TPS}(\text{약 } 334,000\text{번})$
- 평균적으로 사용자 1명은 400명 친구를 갖고,
- 그 가운데 10%가 인근 활성화 상태라고 가정하면
- **1,400만건** 위치 정보 갱신 요청
 - 초당 334,000번 $400\text{명} \times 10\% = 1,400\text{만}$

설계안

- 로드밸런서
 - 앞단에서 트래픽 분산
- RESTful API 서버
 - 통상적인 요청/응답 트래픽 처리

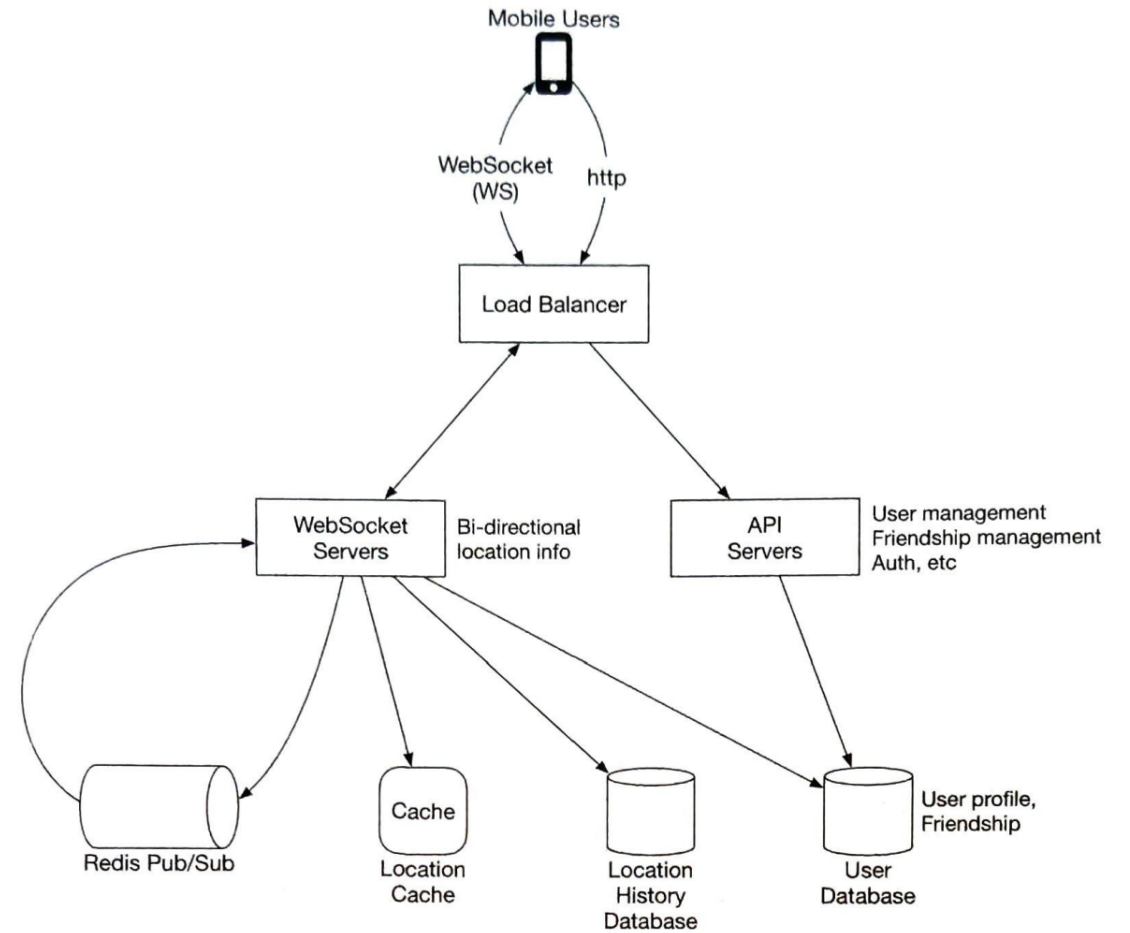
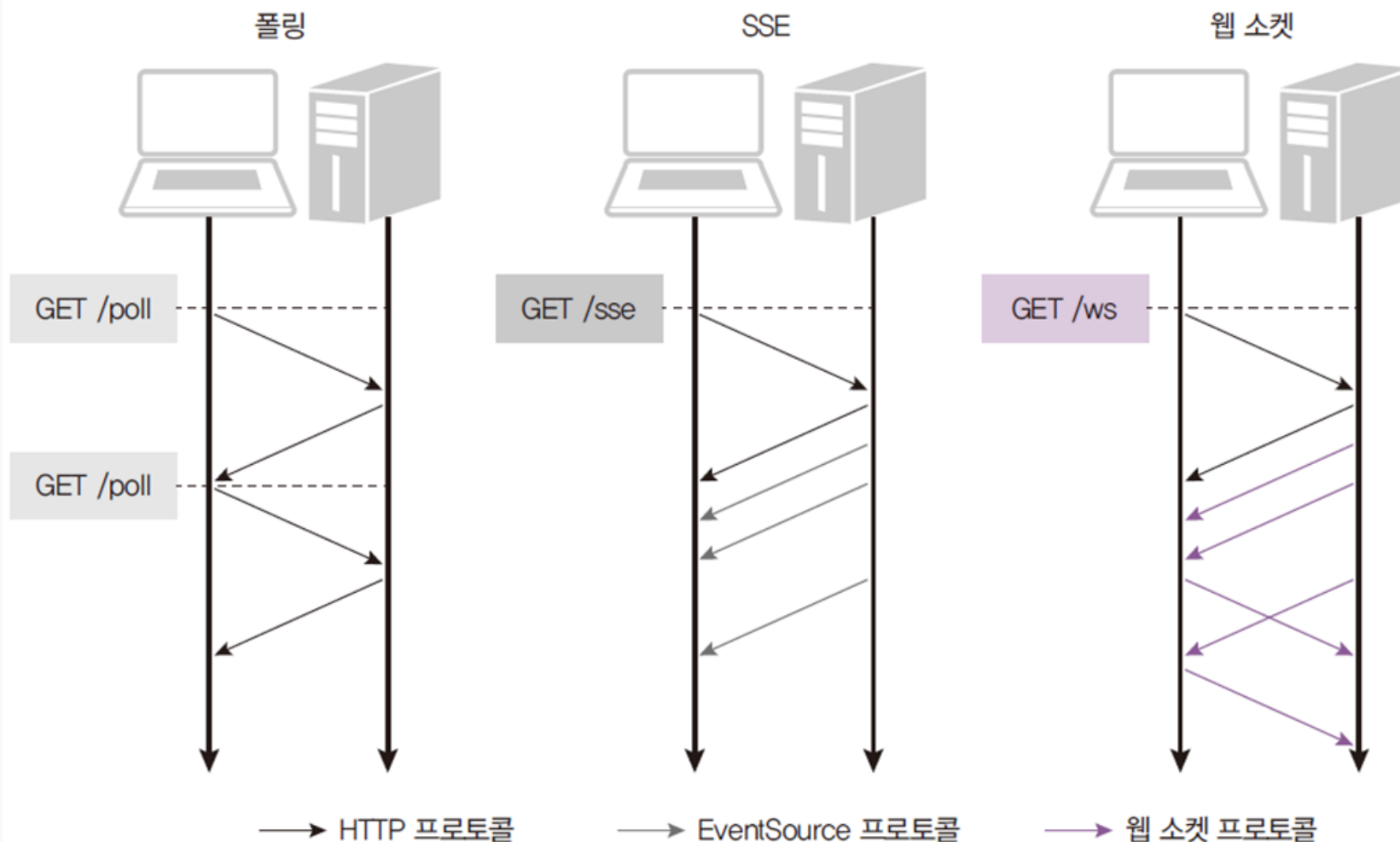


Figure 2.4: High-level design

실시간 데이터 전송 방식

♥ 그림 12-3 폴링 vs. SSE vs. 웹 소켓



Polling	WebSocket	SSE
- Client asks server for data at intervals. - Works over HTTP requests. - Types: short polling, long polling. - Use cases: simple dashboards, status checks. - Pros: easy to build, full client control. - Cons: wastes resources, slower updates. - Examples: checking order status, message refresh.	© Neo Kim + CodeRabbit - Client opens a persistent connection to server. - Enables two-way communication. - Works after HTTP upgrade (101 Switching Protocols). - Use cases: chat apps, multiplayer games, trading apps. - Pros: real-time updates, low latency. - Cons: harder to scale, needs connection management. - Examples: Slack, Discord, live collaboration apps.	- Client opens long HTTP connection. - Server pushes updates when events occur. - Communication is server → client only. - Use cases: live feeds, stock price, notifications. - Pros: simple setup, efficient for streaming updates. - Cons: one-way only, not ideal for interactive apps. - Examples: stock tickers, news feeds, monitoring dashboards.
Download the ultimate prompting cheat sheet: codelab.ai Neo Kim		

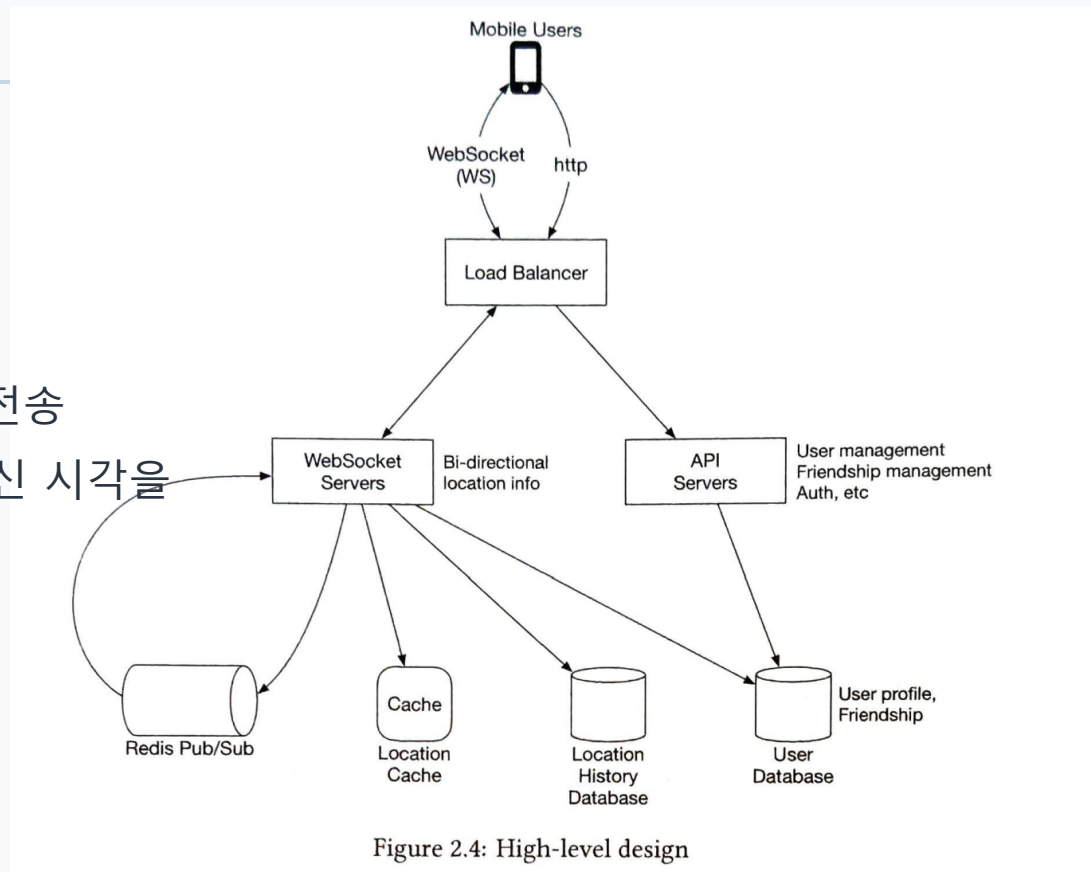
설계안

- WebSocket Server

- 친구 위치 변경을 처리하는 stateful 서버
- 검색 반경 내 친구 위치가 변경되면 해당 내역을 전송
- 모바일 클라이언트가 시작되면, 모든 주변 친구 위치를 전송
- 새로 계산한 거리가 검색 반경 이내면 갱신된 위치와 갱신 시각을 웹소켓 연결을 통해 클라이언트로 보냄

- Redis 위치 정보 캐시

- 활성 상태 사용자의 가장 최근 위치 정보 캐싱



Redis

- Remote(원격)에 위치하고 프로세스로 존재하는 In-Memory 기반의 Dictionary(key-value) 구조 데이터 관리 Server 시스템이다!
- 데이터의 고속 읽기와 쓰기에 최적화

Redis 사용 사례

캐싱

다른 데이터베이스 "앞"에 배치된 Redis는 성능이 뛰어난 인 메모리 캐시를 생성하여 액세스 지연 시간을 줄이고, 처리량을 늘리며, 관계형 또는 NoSQL 데이터베이스의 부담을 줄여줍니다.

세션 관리

Redis는 세션 관리 작업에 매우 적합합니다. Redis를 세션 키에 대한 적절한 TTL과 함께 빠른 키 값 스토어로 사용하면 간단하게 세션 정보를 관리할 수 있습니다. 세션 관리는 주로 게임, 전자 상거래 웹 사이트, 소셜 미디어 플랫폼을 비롯한 온라인 애플리케이션에 필요합니다.

실시간 순위표

Redis Sorted Set 데이터 구조를 사용하면 요소가 목록에 유지되고 점수에 따라 정렬됩니다. 이를 통해 손쉽게 동적 순위표를 생성하여 게임에서 앞서있는 사람이 누구인지 보여주거나, 좋아요를 가장 많이 받은 메시지를 게시하거나, 선두에 있는 사람이 누구인지 보여주려는 다양한 사례에 사용할 수 있습니다.

속도 제한

Redis는 이벤트 속도를 측정하고 필요한 경우 제한할 수 있습니다. 클라이언트의 API 키에 연결된 Redis 카운터를 사용하여 특정 기간 동안 액세스 요청의 수를 세고 한도가 초과되는 경우 조치를 취할 수 있습니다. 속도 제한기는 포럼의 게시물 수를 제한하고, 리소스 사용량을 제한하며, 스팸어의 영향을 억제하는 데 주로 사용됩니다.

대기열

Redis List 데이터 구조를 사용하면 간단한 영구 대기열을 손쉽게 구현할 수 있습니다. Redis List는 자동 작업 및 차단 기능을 제공하므로 신뢰할 수 있는 메시지 브로커 또는 순환 목록이 필요한 다양한 애플리케이션에 적합합니다.

채팅 및 메시징

Redis에서는 패턴 매칭과 더불어 PUB/SUB 표준을 지원합니다. 따라서 Redis를 사용하여 고성능 채팅방, 실시간 코멘트 스트림 및 서버 상호 통신을 지원할 수 있습니다. 또한 PUB/SUB를 사용하여 게시된 이벤트를 기반으로 작업을 트리거할 수 있습니다.



설계안

- 위치 이동 이력 데이터베이스
- 사용자 데이터베이스

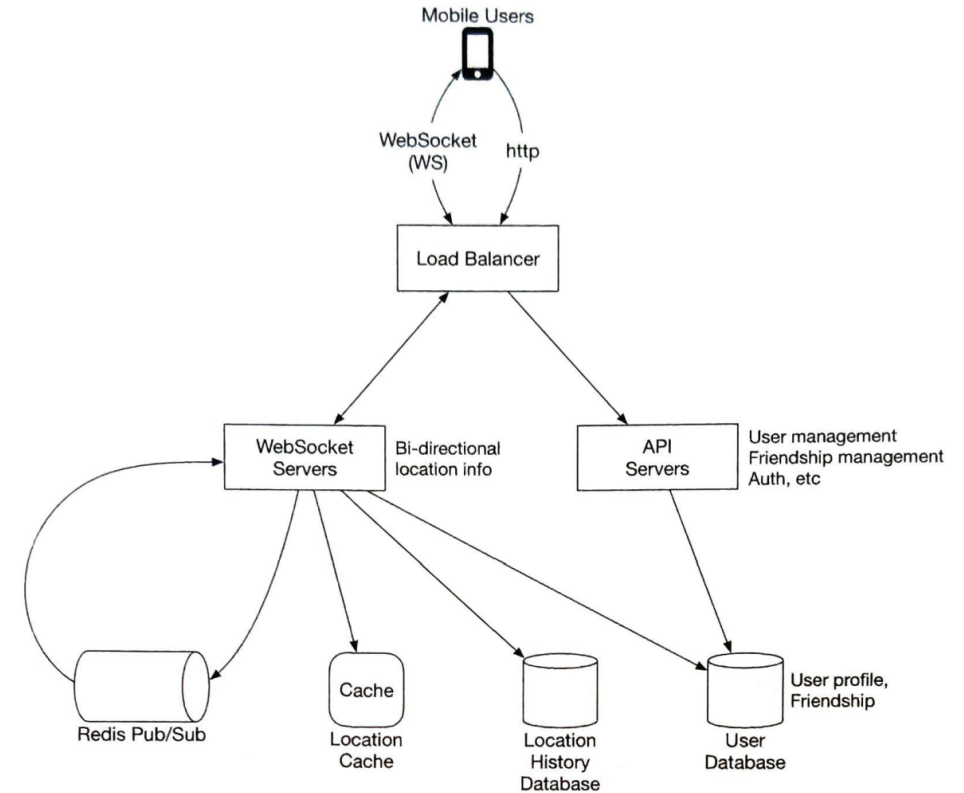


Figure 2.4: High-level design

Redis Pub/Sub

- 특정 사용자의 친구 각각과 연결된 웹소켓 핸들러는 해당 채널의 구독자로 설정된다

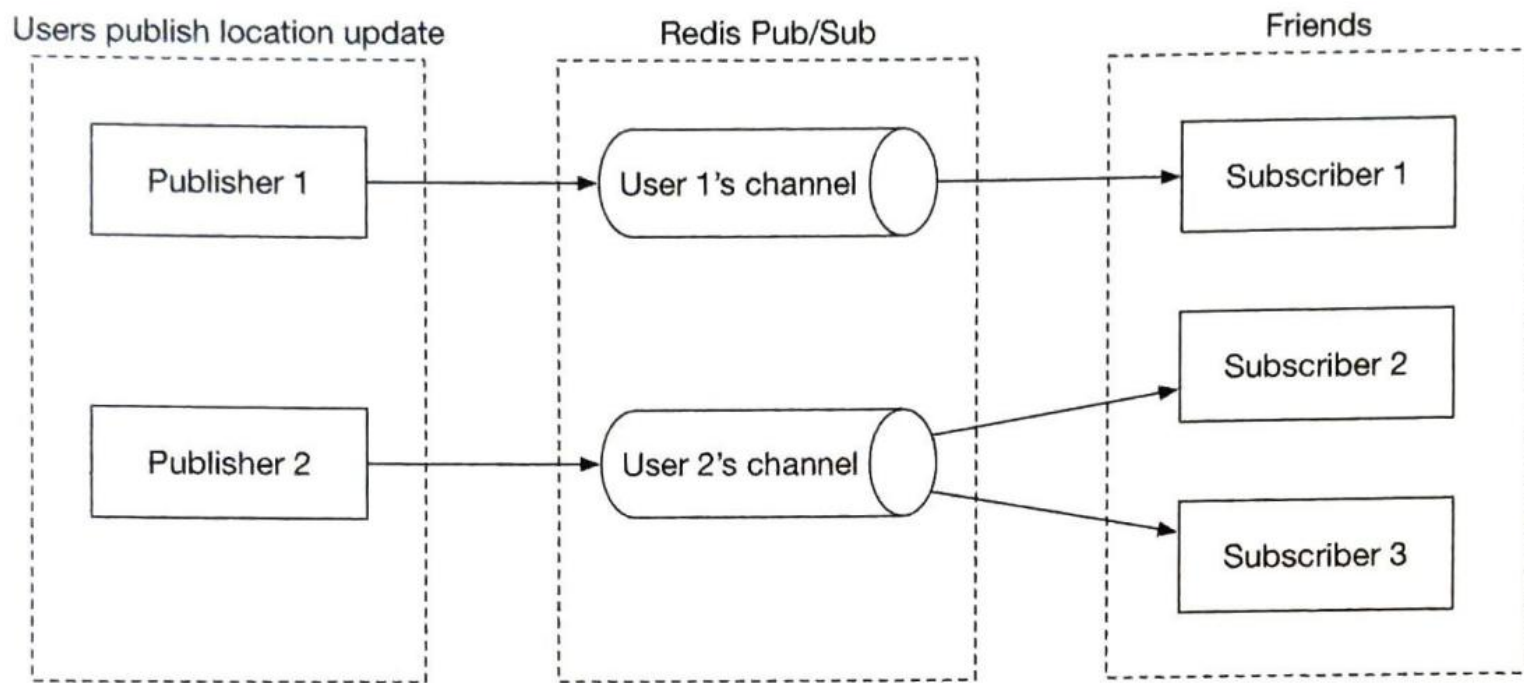


Figure 2.6: Redis Pub/Sub

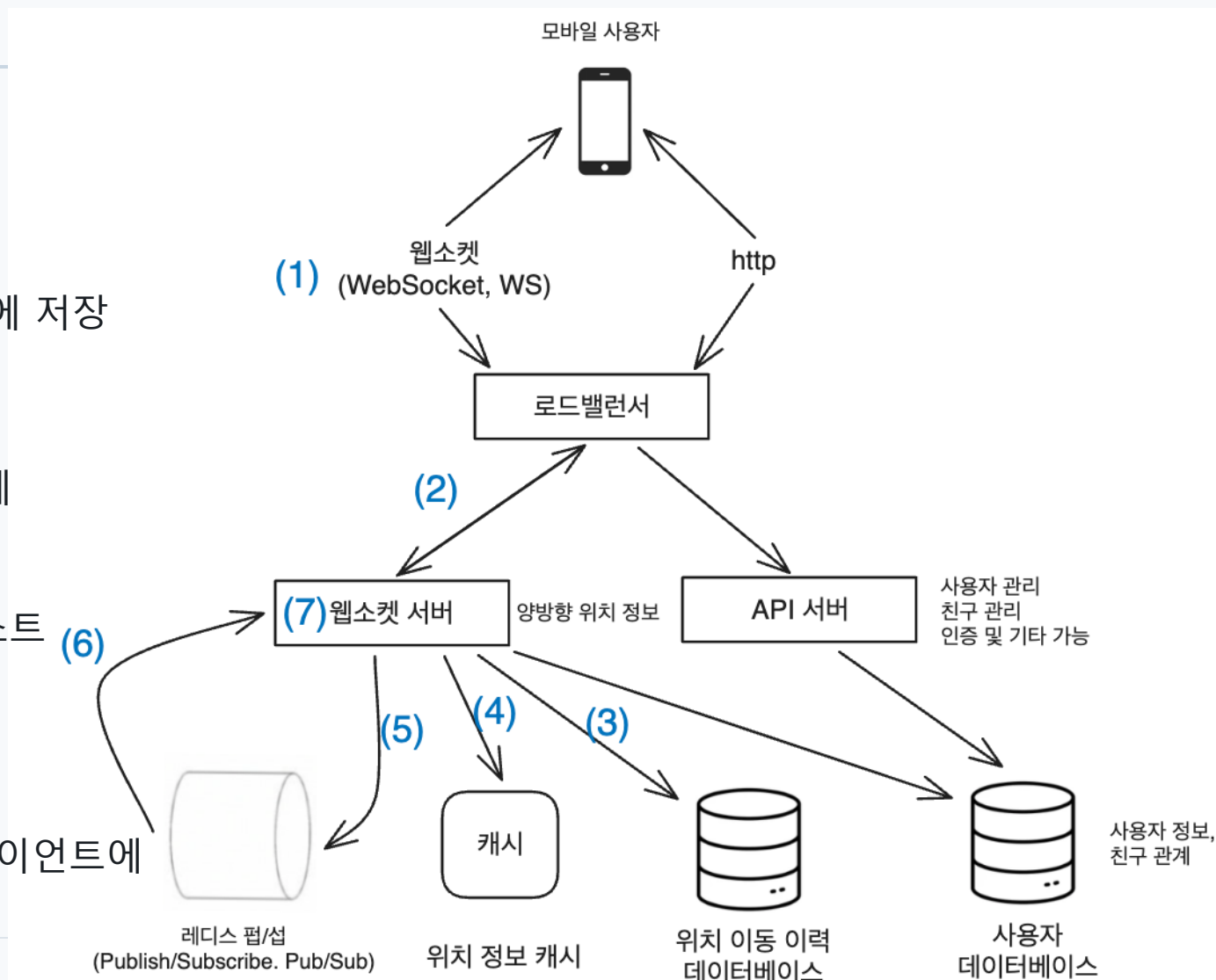
채널: 라디오 주파수

Publisher: 방송 송출

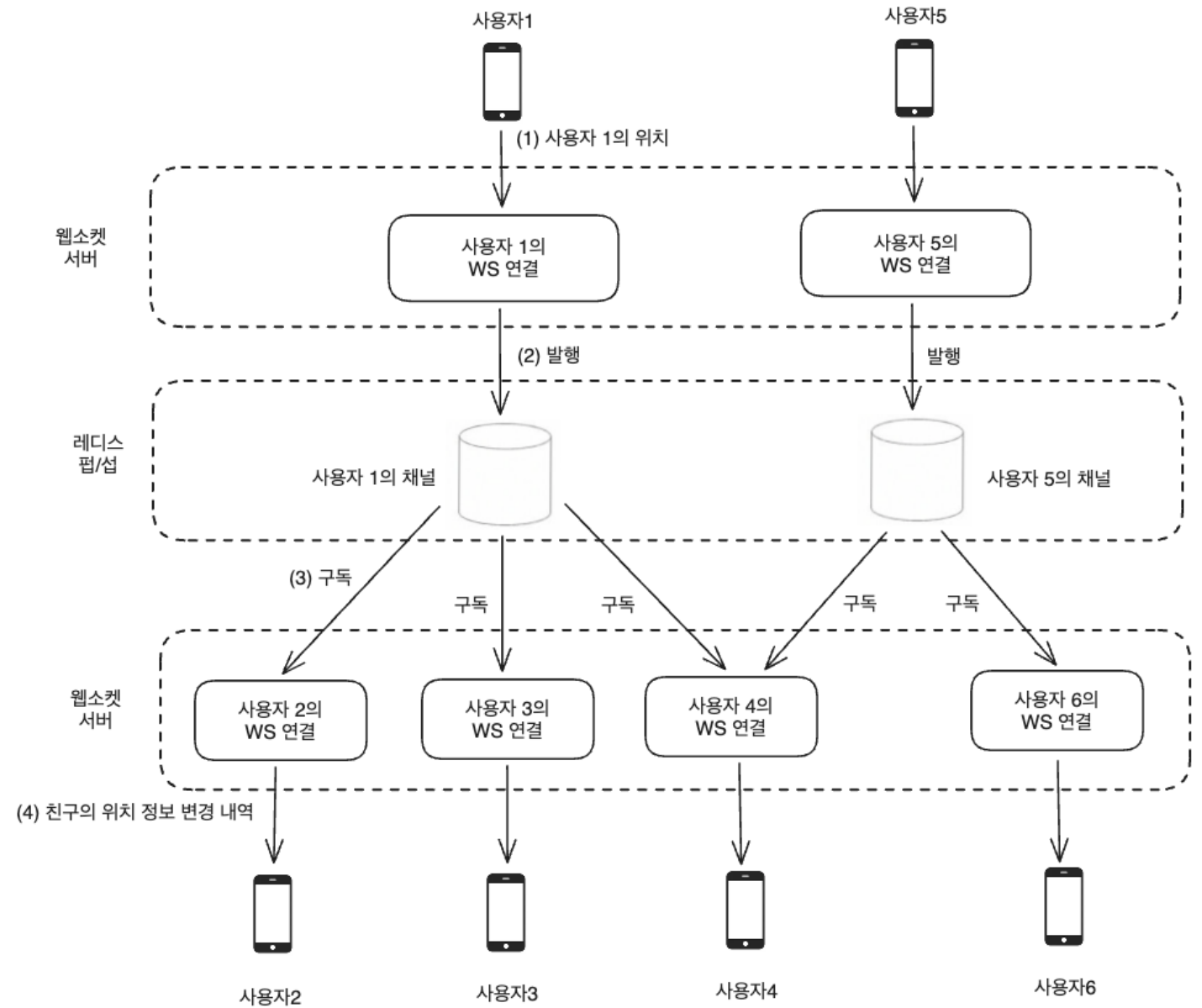
Subscriber: 라디오 켜고 채널 맞추기

주기적 위치 갱신

1. 클라이언트 위치 변경 사실을 로드밸런서에 전송
2. 로드밸런서는 그 위치 변경 내역을 웹소켓으로 전송
3. 웹소켓 서버는 해당 이벤트를 위치 이동 데이터베이스에 저장
4. 웹소켓 서버는 캐시에 저장. TTL도 갱신
5. 웹소켓 서버는 레이스 펌/섭 서버에 해당 사용자 채널에 위치 발행. 3~5는 병렬로 수행
6. 레디스 펌/섭 모든 구독자(온라인 친구)에게 브로드캐스트
7. 새 위치를 보낸 사용자와 메시지를 받은 사용자의 사이 거리 새로 계산
8. 7에서 계산한 거리가 검색 반경에 넘지 않으면 각 클라이언트에 정보를 전송하고, 넘으면 전송 안함



주기적 위치 갱신



위치 정보 캐시

위치 정보 저장에 DB를 사용하지 않는 이유

- 사용자 위치는 하나만 보관하기에 충분
- 레디스는 읽기 및 쓰기 연산 속도가 엄청나게 빠르므로 해당 목적에 적합
- 주변 친구 기능이 활용하는 위치 정보에 대해서는 영속성을 보장할 필요가 없다. (일회성) + TTL 존재

Key	Value
user_id	{위도, 경도, 시각}

위치 이동 이력 데이터베이스

[user_id, latitude, longitude, timestamp]

요구사항

막대한 쓰기 연산 부하

수평적 규모 확장 가능

카산드라 or 샤딩 가능한 어떤 DB



고성능, 고가용성, 수평 확장성을 목표로 설계된 분산형 NoSQL 데이터베이스입니다.
Cassandra의 아키텍처는 전통적인 RDBMS와는 다르게 Masterless 구조를 채택하고 있어,
모든 노드가 동등한 역할을 수행합니다.

주요 특징

- Peer-to-Peer 기반 : 모든 노드는 서로 대등하며, 데이터는 클러스터 전체에 분산 저장됩니다.
- 무중단 운영 : 일부 노드가 실패해도 전체 시스템은 정상적으로 동작합니다.
- 튜닝 가능한 일관성 : 사용자가 읽기/쓰기 요청에 대해 원하는 일관성 수준을 선택할 수 있습니다.

상세 설계

레디스 펌/셋 서버가 얼마나 필요?

메모리 사용량

- 필요한 채널의 수: 1억 개(10억 사용자의 10%)
- 모든 채널을 저장하는 데 필요한 메모리: 200GB(1억 *20바이트* 100명의 친구 / 10^9)

한 사용자의 활성화 상태 친구 가운데 **100명이 주변 친구 기능을 사용**한다고 가정
구독자 한 명을 추적하기 위해 내부 해시 테이블과 연결 리스트에 **20바이트 상당의 포인터**들을 저장해야 함
100GB 메모리를 설치할 수 있는 최신 서버를 사용할 경우, 모든 채널을 보관하는 데 레디스 펌/셋 서버 2대면 된다.

CPU 사용량

- 전송 위치 정보 업데이트 양: 초당 1400만 건
- 필요한 레디스 서버의 수: $1400\text{만} / 100,000 = 140\text{대}$ (서버 한 대로 감당 가능한 구독자의 수는 100,000이라고 가정)

레디스 펌/셋 서버의 병목은 CPU 사용량이다.

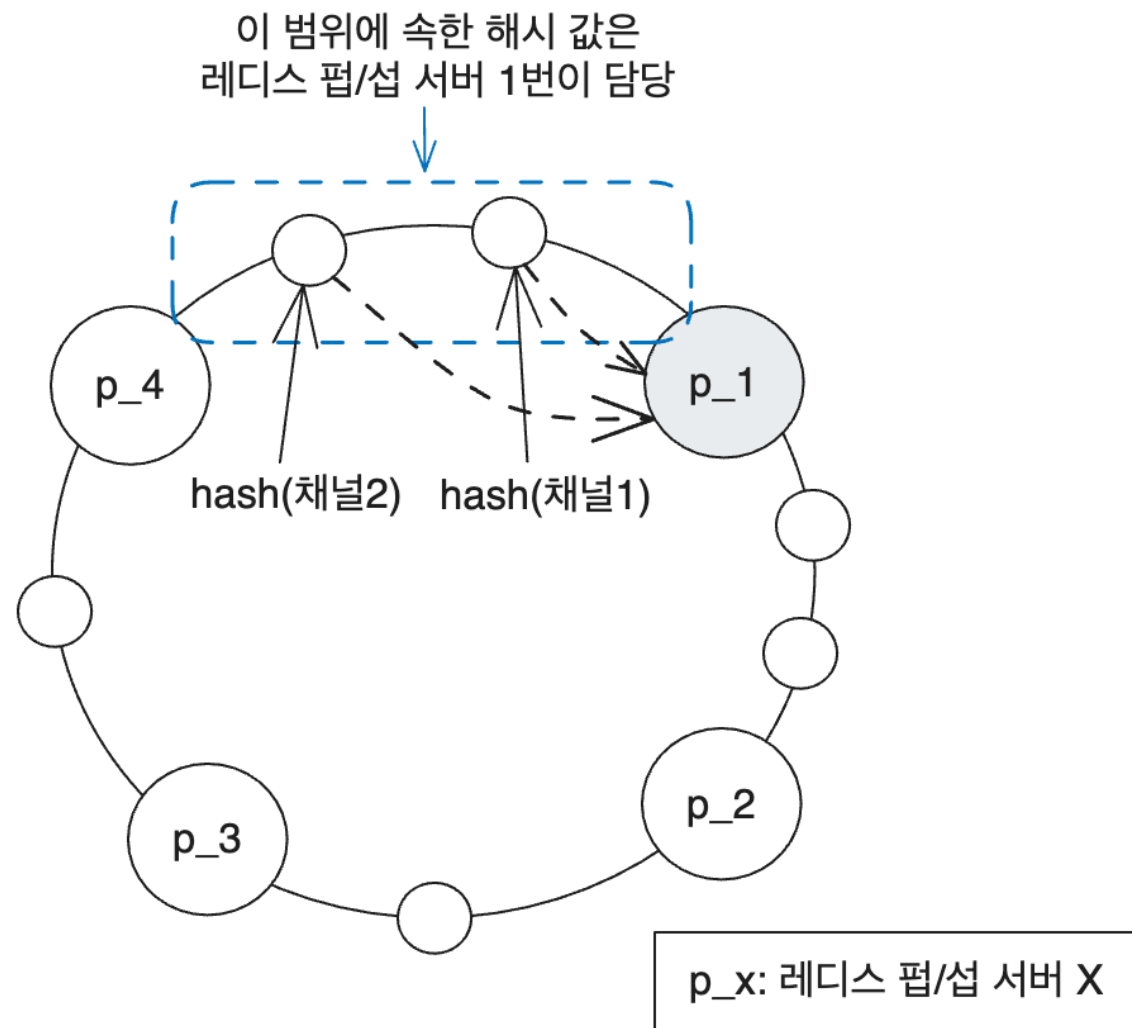
분산 레디스 펌/셋 클러스터

- 서비스 탐색 컴포넌트

- Zookeeper, etcd

[서비스 A] → [서비스 B]를 호출하고 싶은데,
서비스 B가 어디 떠있는지 모름

[서비스 B 인스턴스들] → 뜰 때 자기 위치를 등록
→ [etcd/ZooKeeper] ← 조회 → [서비스 A]

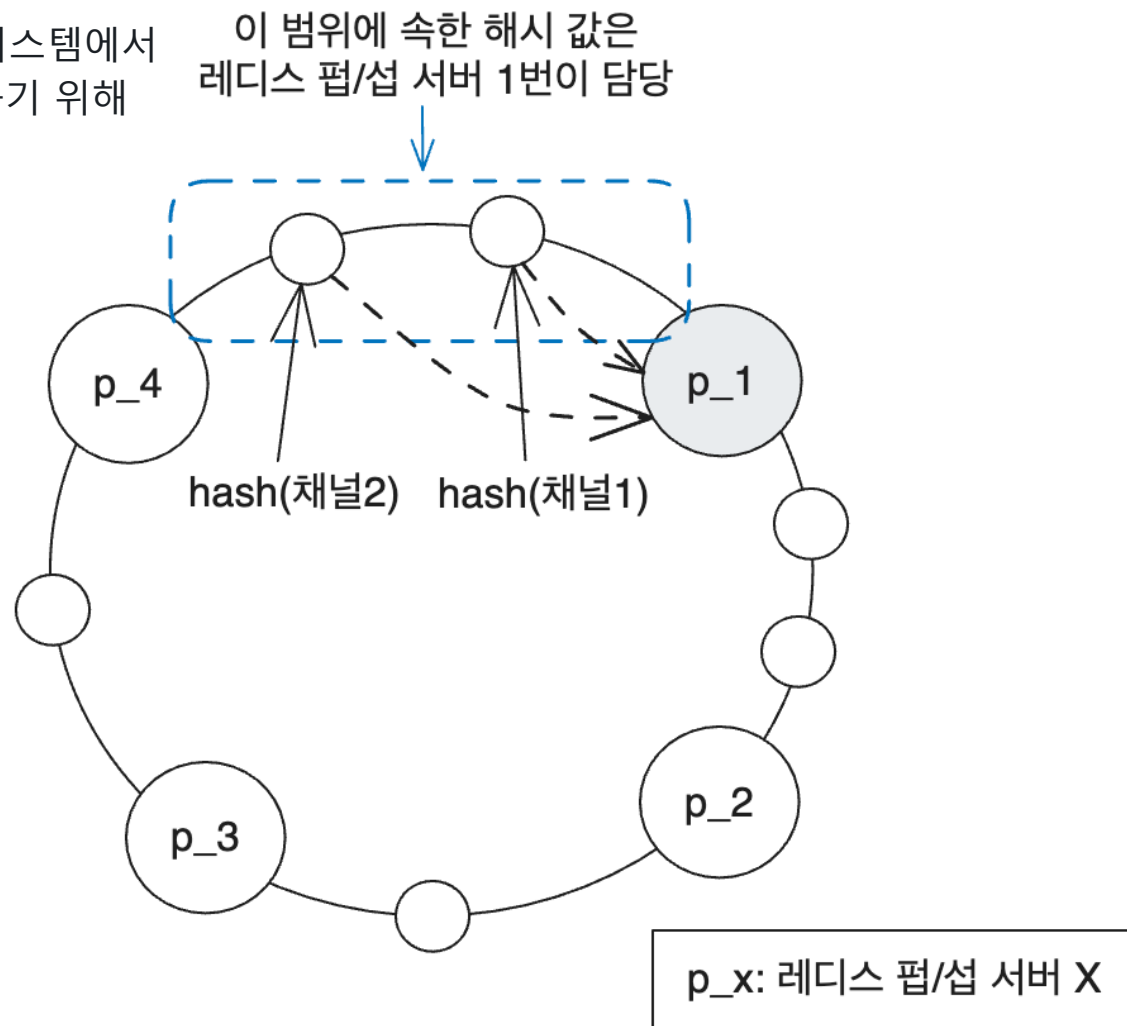


분산 레디스 펌/셋 클러스터

안전 해시 또는 **Consistent Hashing**(일명 링 해시, 해시 링)는 주로 분산 시스템에서 데이터 분산과 서버 확장/축소 시 데이터 재분배 문제를 효율적으로 해결하기 위해 사용되는 해싱 기법

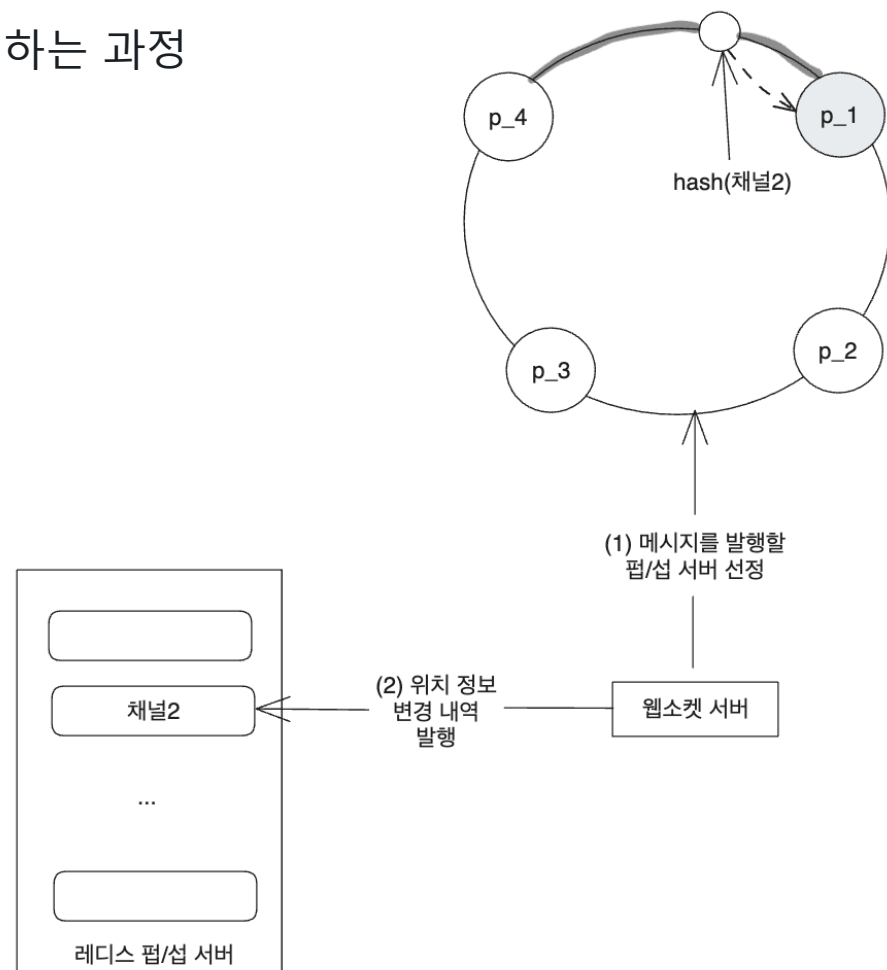
1. 해시 공간을 링(원형)으로 표현
2. 서버(노드)와 데이터의 해시값 할당
3. 서버 추가/제거 시 동작
 1. 서버 추가: 새로운 서버가 링에 추가되면, 해당 서버가 담당하는 구간의 데이터 일부만 새 서버로 이동
 2. 서버 제거: 서버가 제거되면, 그 서버가 담당하던 구간의 데이터만 인접한 서버로 이동
전체 데이터 중 일부만 이동하므로, 시스템 확장/축소 시 효율적
4. 부하 분산 및 **가상 노드**

실제 서버 수가 적을 때 데이터가 한쪽에 몰리는 현상을 방지하기 위해, 하나의 서버를 여러 개의 가상 노드로 만들어 링에 여러 번 배치할 수 있다.



분산 레디스 펌/셋 클러스터

웹소켓 서버가 특정 사용자 채널에 위치 정보 변경 내역을 발행하는 과정



분산 레디스 펌/셋 클러스터

유상태 vs 무상태

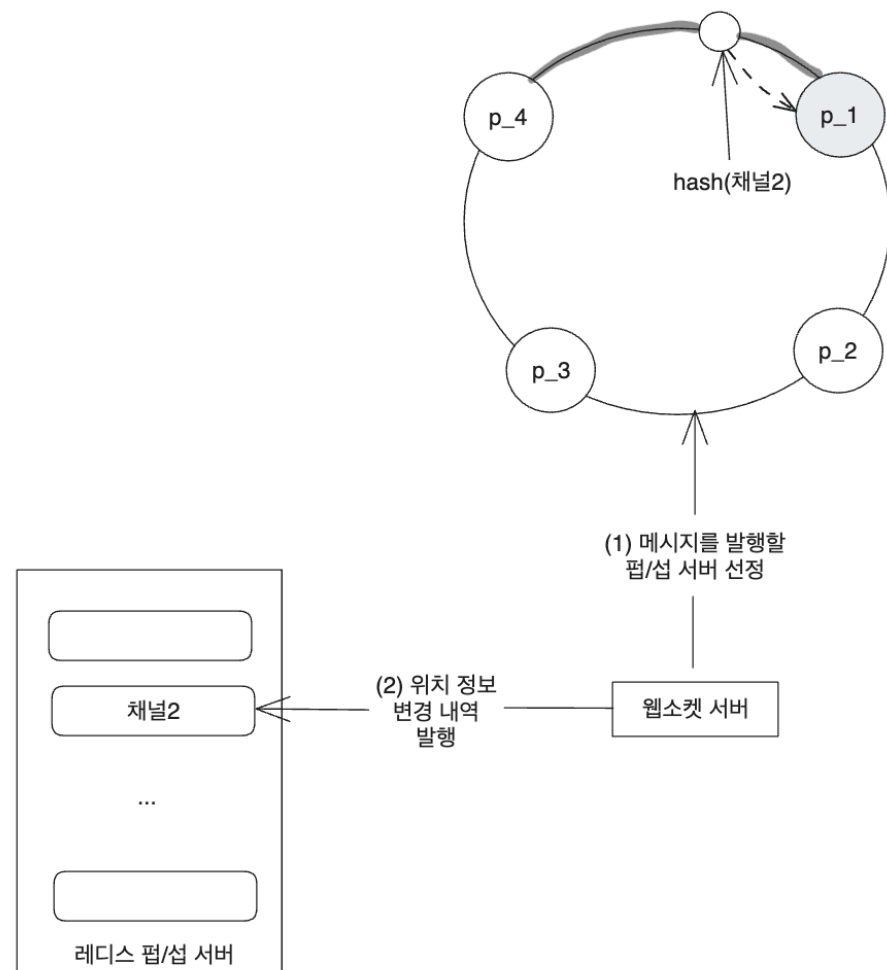
무상태

펌/셋 채널에 전송되는 메시지는 지속적으로 보관되지 않음
전송 후 바로 삭제
구독자가 없으면 그냥 지워짐

유상태

채널에 대한 상태 정보를 보관
채널이 다른 서버로 이동하면 모든 구독자들에게 알려야 함
기존 구독자는 이전 채널의 구독을 해지하고, 새 서버에 마련된 대체 채널 구독해야 하기 때문

-> 이 관점에서 유상태로 취급하는 것이 바람직



친구 추가/삭제

큰 앱의 일부 기능이므로 친구 추가/삭제 시 호출될 콜백으로 등록

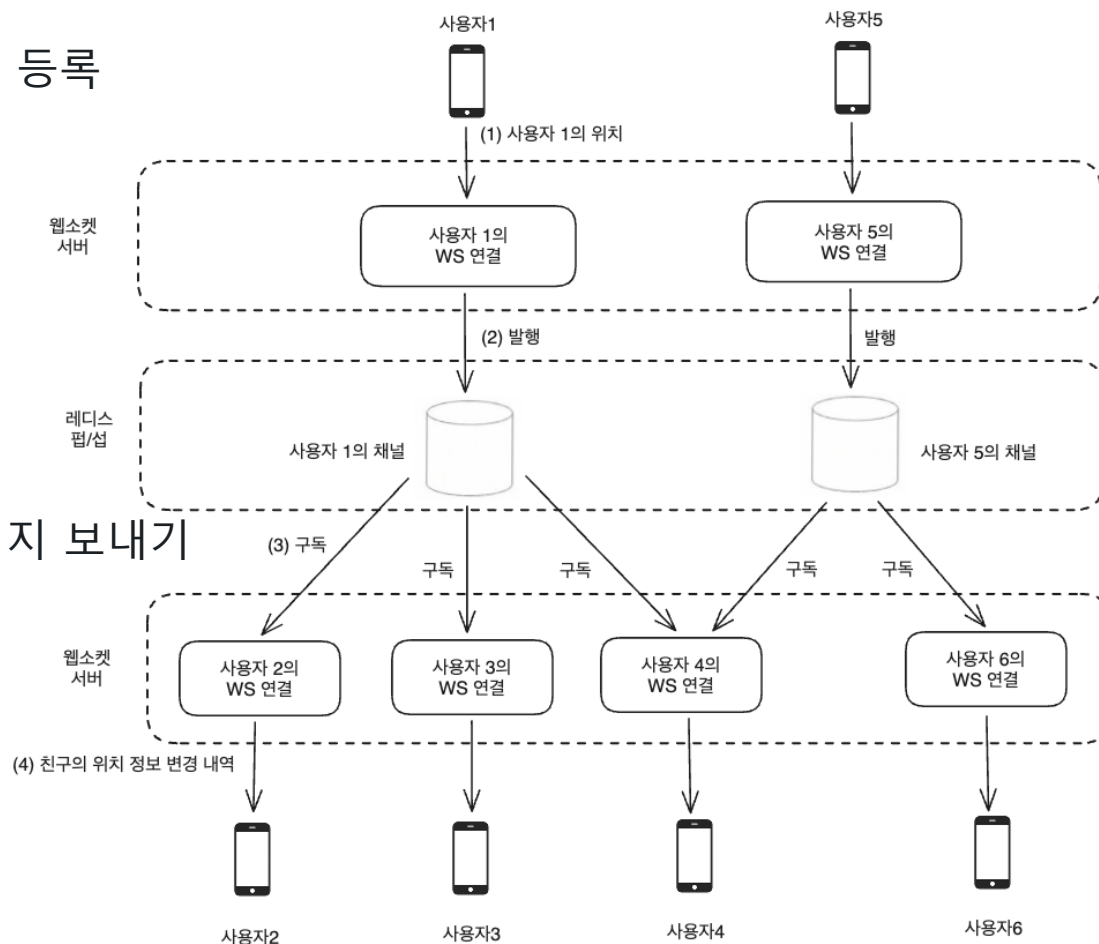
친구 추가

웹소켓 서버에게 친구 추가 사실 알리기

새 친구의 펌/섭 채널 구독

친구 삭제

웹소켓 서버로 해당 친구의 펌/섭 채널 구독 취소하라는 메시지 보내기



친구가 많은 사용자

친구 관계는 양방향, 팔로어 모델 배제

친구들의 위치 변경에서 오는 부하 – 각 웹소켓 서버가 나눠서 처리

많은 친구를 둔 사용자의 채널이 존재하는 펌/셋 서버

조금 더 많은 부하 감당

하지만 클러스터 안에 100대 넘는 펌/셋 서버

헤비 유저들의 채널도 분산되어 있음

주변의 임의 사용자

지오 해시

격자마다 채널을 만들고 격자 내의 사용자는 할당된 채널 구독

위치 변경되면 해당 지오해시 채널에 새 위치 전송

근방에 있는 사용자 중 해당 채널 구독자는 위치 변경 메시지 수신

격자 경계를 잘 처리하기 위해서는 주변 지오해시 격자 채널도 구독

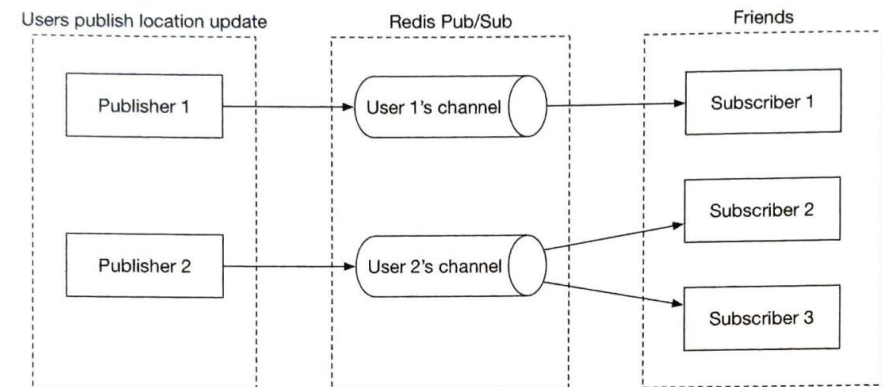
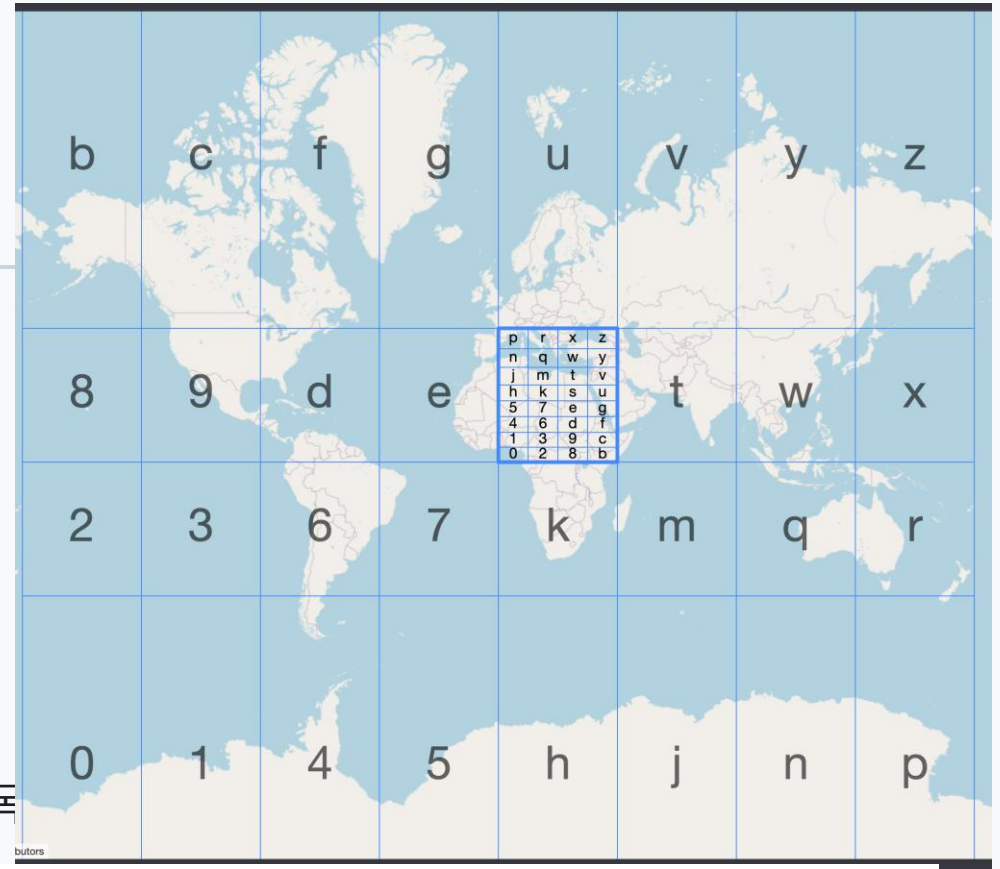


Figure 2.6: Redis Pub/Sub

Redis vs Kafka

	Kafka	Redis
메시지 저장 여부	토픽에 메시지를 일정 주기동안 저장할 수 있음	메시지가 Subscriber 에게 전송되면서 채널에서 바로 삭제됨
메시지 수신 방식	Consumer 가 polling 방식으로 Topic 에서 메시지를 가져감	채널에서 Subscriber 들에게 메시지를 push
전송보장	ack 옵션에 따라 보장 가능	메시지를 수신했는지 따로 확인하지 않음
지연시간	메시지에 대해 ack 옵션에 따라 수신 확인 등의 작업이 있어 redis 보다는 비교적 느림(그래도 빠르다)	In-Memory 기반으로 메시지 전송 후 수신 확인같은 별도의 작업이 없어 Kafka 보다 비교적 가볍고 빠름

